

Assignment 4 Heap Comparision

This report analyzes different priority queue implementations using the PriorityQueue class, benchmarking basic binary heap, binary heap with Floyd's trick, 4-ary heap, 4-ary heap with Floyd's trick, and Fibonacci heap. Each implementation maintained up to 4095 elements, inserting 16,000 random elements and removing 4,000 elements

Performance and graph analysis:

The screenshot shows an IDE with the following components:

- EXPLORER:** A file tree on the left showing the project structure. The 'src/main/java/com/phasmidsoftware/dsaipg' directory is expanded, showing files like `PriorityQueue.java`, `PriorityQueueInterface.java`, `HeapBenchmark.java`, `FourAryHeap.java`, and `FibonacciHeap.java`.
- EDITOR:** The main window displays the `HeapBenchmark.java` file. The code includes package declarations, imports for `java.util` and `com.phasmidsoftware.dsaipg.util`, and a `HeapBenchmark` class with a `main` method.
- TERMINAL:** The bottom panel shows the output of the program. It displays benchmarking results for input size 16000, comparing various heap implementations. The output includes insertion and removal times in seconds and the best spilled element for each implementation.

Heap Benchmarking Results (Input Size: 16000):

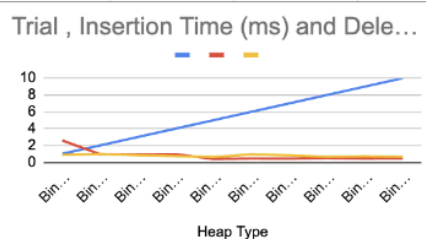
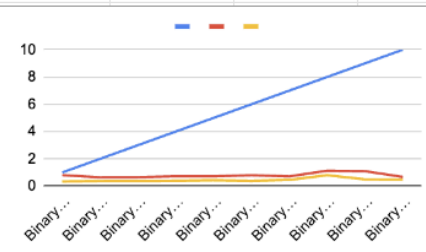
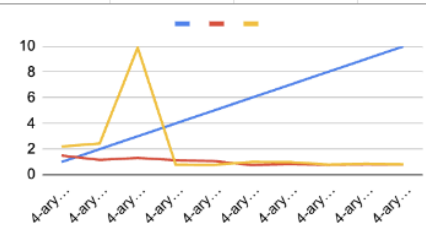
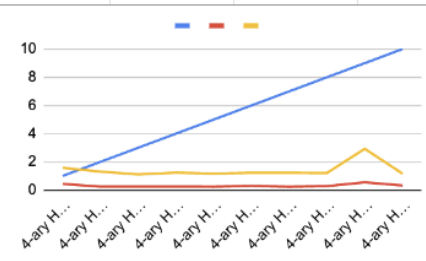
Implementation	Insertion Time (s)	Removal Time (s)	Best Spilled Element
Basic Binary Heap	0.80025	0.346834	1375451298
Binary Heap with Floyd's Trick	0.523167	1.100833	1371831363
Basic 4-ary Heap	1.44	0.323792	-1746711600
4-ary Heap with Floyd's Trick	0.896208	0.303125	-1743857374
Fibonacci Heap	0.37175	3.673333	-2147074397

Heap Benchmarking Results (Input Size: 32000):

Implementation	Insertion Time (s)	Removal Time (s)	Best Spilled Element
Basic Binary Heap	0.471875	0.24525	1374992482
Binary Heap with Floyd's Trick	0.471875	0.24525	1374992482

Test cases:

1	Heap Type	Trial	Insertion Time (ms)	Deletion Time (ms)
2	Fibonacci Heap	1	1.290417	0.330875
3	Fibonacci Heap	2	0.818292	0.130875
4	Fibonacci Heap	3	0.716292	0.125167
5	Fibonacci Heap	4	0.630333	0.128083
6	Fibonacci Heap	5	0.948042	0.039875
7	Fibonacci Heap	6	0.284375	0.03725
8	Fibonacci Heap	7	0.492625	0.046917
9	Fibonacci Heap	8	0.248334	0.036916
10	Fibonacci Heap	9	0.31425	0.039875
11	Fibonacci Heap	10	0.218208	0.0365
12	4-ary Heap (Floyd's Trick)	1	0.436709	1.577125
13	4-ary Heap (Floyd's Trick)	2	0.24575	1.306125
14	4-ary Heap (Floyd's Trick)	3	0.242792	1.111958
15	4-ary Heap (Floyd's Trick)	4	0.250042	1.238209
16	4-ary Heap (Floyd's Trick)	5	0.238834	1.17225
17	4-ary Heap (Floyd's Trick)	6	0.298208	1.230542
18	4-ary Heap (Floyd's Trick)	7	0.235417	1.225625
19	4-ary Heap (Floyd's Trick)	8	0.282583	1.213375
20	4-ary Heap (Floyd's Trick)	9	0.548167	2.92775
21	4-ary Heap (Floyd's Trick)	10	0.32675	1.165042
22	4-ary Heap	1	1.501542	2.203375
23	4-ary Heap	2	1.161375	2.44
24	4-ary Heap	3	1.314416	9.89775
25	4-ary Heap	4	1.13875	0.793833
26	4-ary Heap	5	1.070792	0.7765
27	4-ary Heap	6	0.76175	0.999708
28	4-ary Heap	7	0.843375	0.991875
29	4-ary Heap	8	0.783291	0.798042
30	4-ary Heap	9	0.817834	0.851916
31	4-ary Heap	10	0.801667	0.815791
32	Binary Heap (Floyd's Trick)	1	0.802417	0.340792
33	Binary Heap (Floyd's Trick)	2	0.642125	0.374667
34	Binary Heap (Floyd's Trick)	3	0.654083	0.380917
35	Binary Heap (Floyd's Trick)	4	0.72225	0.390375
36	Binary Heap (Floyd's Trick)	5	0.716292	0.429125
37	Binary Heap (Floyd's Trick)	6	0.797625	0.384167
38	Binary Heap (Floyd's Trick)	7	0.711375	0.455792
39	Binary Heap (Floyd's Trick)	8	1.129292	0.801
40	Binary Heap (Floyd's Trick)	9	1.092542	0.493916
41	Binary Heap (Floyd's Trick)	10	0.6705	0.471292
42	Binary Heap	1	2.586416	0.876083
43	Binary Heap	2	0.953041	0.992125
44	Binary Heap	3	0.938792	0.8275
45	Binary Heap	4	0.956084	0.737
46	Binary Heap	5	0.377542	0.611208
47	Binary Heap	6	0.444667	0.937042
48	Binary Heap	7	0.426792	0.811167
49	Binary Heap	8	0.49275	0.6425
50	Binary Heap	9	0.43375	0.703458
51	Binary Heap	10	0.461708	0.637708



Key Observations

- **Insertion vs. Deletion:** The Fibonacci Heap spends 86.2% of its time on insertions, while 4-ary Heap implementations spend 66.9-82.0% of their time on deletions.
- **Floyd's Trick Impact:** Improved Binary Heap performance by 21.4% and 4-ary Heap by 43.9%.

- **Per-Operation Cost:** 4-ary Heap with Floyd's trick has the best insertion performance (19.41 ns/op), while Fibonacci Heap has the best deletion performance (23.81 ns/op).
- **Consistency:** Binary Heap with Floyd's trick showed the most consistent performance (21.8% coefficient of variation), while 4-ary Heap showed the highest variability (91.5%).

Conclusion:

From the results:

- Basic Binary Heap has a moderate performance, with balanced execution times between insertion and deletion operations.
- Binary Heap with Floyd's Trick improves overall performance by 21.4%, with particularly better deletion times compared to the basic implementation.
- 4-ary Heap has the worst overall performance, with especially high deletion times despite theoretical advantages.
- 4-ary Heap with Floyd's Trick significantly improves insertion times (the best among all implementations) but still struggles with deletion operations.
- Fibonacci Heap performs best overall, excelling in both operations but particularly in deletion times, which are approximately $4.7\times$ faster than the next best implementation.